

# 📡 СИГНАЛЫ И СЛОТЫ ДЛЯ VideoEditor Pro

## 🏗️ СТРУКТУРА КЛАССОВ

...

Timeline (timeline.h)  
├─ управляет клипами  
├─ хранит TimelineClip  
└─ координирует FFmpegHelper

FFmpegHelper (ffmpeghelper.h)  
├─ работает с видео через FFmpeg  
├─ декодирует кадры  
├─ извлекает аудио  
└─ получает метаданные  
...

---

## 🕒 Timeline.h - ПОЛНЫЙ СПИСОК

```
```cpp
#ifndef TIMELINE_H
#define TIMELINE_H

#include <QObject>
#include <QList>
#include "timelineclip.h"

class Timeline : public QObject {
    Q_OBJECT

    // ===== PROPERTIES для QML =====
    Q_PROPERTY(double currentTime READ currentTime WRITE setCurrentTime NOTIFY
currentTimeChanged)
    Q_PROPERTY(double totalDuration READ totalDuration NOTIFY totalDurationChanged)
    Q_PROPERTY(int clipCount READ clipCount NOTIFY clipsChanged)
    Q_PROPERTY(bool isPlaying READ isPlaying NOTIFY playbackStateChanged)

public:
    explicit Timeline(QObject *parent = nullptr);

    // Геттеры
    double currentTime() const { return m_currentTime; }
    double totalDuration() const;
    int clipCount() const { return m_clips.size(); }
    bool isPlaying() const { return m_isPlaying; }

public slots:
    // ===== УПРАВЛЕНИЕ КЛИПАМИ =====
    // Все слоты доступны из QML через cppTimeline.methodName()

    bool addClip(const QString& filepath, int trackIndex, double startTime);
    bool removeClip(int index);
    bool moveClip(int index, int newTrackIndex, double newStartTime);
    bool splitClip(int index, double splitTime);
    bool trimClip(int index, double newTrimStart, double newTrimEnd);
    bool applyEffect(int index, const QString& effectName, double value);
};

#endif
```

```

// Получить клипы для дорожки (для QML Repeater)
QVariantList getClipsForTrack(int trackIndex);

// Получить текущий клип под playhead
QVariantMap getCurrentClip();

// ===== ВОСПРОИЗВЕДЕНИЕ =====
void setCurrentTime(double time);
void play();
void pause();
void stop();
void seek(double time);

// ===== ПРОЕКТ =====
bool saveProject(const QString& filepath);
bool loadProject(const QString& filepath);
bool renderToFile(const QString& outputPath);

signals:
// ===== СИГНАЛЫ ДЛЯ QML =====

// Изменения timeline
void clipsChanged(); // Список клипов изменился
void clipAdded(int index); // Клип добавлен
void clipRemoved(int index); // Клип удалён
void clipModified(int index); // Клип изменён (trim/move/effects)
void totalDurationChanged(); // Изменилась общая длительность

// Воспроизведение
void currentTimeChanged(); // Playhead подвинулся
void playbackStateChanged(); // play/pause/stop

// Текущий клип под playhead (для VideoPlayer)
void currentClipChanged(QString filepath, double position);

// Рендеринг
void renderStarted();
void renderProgress(int percent);
void renderFinished(bool success);
void renderError(QString message);

// Ошибки
void errorOccurred(QString message);

// Метаданные от FFmpeg (асинхронно)
void clipDurationReady(int clipIndex, double duration);
void clipThumbnailReady(int clipIndex, QString thumbnailPath);

private:
QList<TimelineClip> m_clips;
double m_currentTime;
bool m_isPlaying;

void sortClips();
void updateCurrentClip(); // Найти клип под playhead и emit currentClipChanged
};

#endif

```

---

##  FFmpegHelper.h - ПОЛНЫЙ СПИСОК

```cpp

```
#ifndef FFMPEGHELPER_H
#define FFMPEGHELPER_H
```

```
#include <QObject>
#include <QString>
#include <QImage>
```

```
// Этот класс работает с FFmpeg
// НЕ экспортируется в QML напрямую - используется только внутри Timeline
```

```
class FFmpegHelper : public QObject {
    Q_OBJECT
```

```
public:
```

```
    explicit FFmpegHelper(QObject *parent = nullptr);
    ~FFmpegHelper();
```

```
public slots:
```

```
    // ===== МЕТАДААННЫЕ =====
```

```
    // Эти слоты вызываются из Timeline::addClip()
```

```
    // Получить длительность видео (асинхронно)
```

```
    void getDuration(const QString& filepath);
```

```
    // Извлечь аудио в отдельный файл
```

```
    void extractAudio(const QString& videoPath, const QString& outputPath);
```

```
    // Создать миниатюру (кадр из видео)
```

```
    void generateThumbnail(const QString& filepath, double time, const QString&
outputPath);
```

```
    // ===== ДЕКОДИРОВАНИЕ =====
```

```
    // Эти слоты вызываются когда нужен конкретный кадр
```

```
    // Получить кадр в указанной позиции
```

```
    void getFrame(const QString& filepath, double time);
```

```
    // ===== РЕНДЕРИНГ =====
```

```
    // Вызывается из Timeline::renderToFile()
```

```
    // Рендерить timeline в файл
```

```
    void renderTimeline(const QList<TimelineClip>& clips, const QString&
outputPath);
```

```
signals:
```

```
    // ===== СИГНАЛЫ =====
```

```
    // Метаданные готовы
```

```
    void durationReady(QString filepath, double duration);
```

```
    void audioExtracted(QString videoPath, QString audioPath);
```

```
    void thumbnailReady(QString filepath, QString thumbnailPath);
```

```
    // Кадр декодирован
```

```

void frameReady(QString filepath, double time, QImage frame);

// Рендеринг
void renderProgress(int percent);
void renderFinished(bool success);
void renderError(QString message);

// Ошибки FFmpeg
void errorOccurred(QString message);

private:
    // FFmpeg контекст (будет реализован позже)
    void* m_formatContext;
    void* m_codecContext;
};

#endif
```

---

## 📎 ПОДКЛЮЧЕНИЕ В main.cpp

```cpp
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext>
#include <QQuickStyle>
#include <QQuickWindow>
#include "timeline.h"
#include "ffmpeghelper.h"

int main(int argc, char *argv[]) {
    // OpenGL setup
    qputenv("QSG_RHI_BACKEND", "opengl");
    QQuickWindow::setGraphicsApi(QSGRendererInterface::OpenGL);
    QQuickStyle::setStyle("Basic");

    QGuiApplication app(argc, argv);
    QQmlApplicationEngine engine;

    // ===== СОЗДАЁМ C++ ОБЪЕКТЫ =====
    Timeline timeline;
    FFmpegHelper ffmpeg;

    // ===== СВЯЗЫВАЕМ Timeline ↔ FFmpegHelper =====

    // Когда Timeline нужна длительность → FFmpeg получает её
    QObject::connect(&timeline, &Timeline::clipAdded,
                    &ffmpeg, [&ffmpeg, &timeline](int index) {
        // Получаем клип
        QVariantList clips = timeline.getClipsForTrack(0); // Упрощение
        if (index < clips.size()) {
            QString filepath = clips[index].toMap()["filepath"].toString();
            ffmpeg.getDuration(filepath);
        }
    });

    // Когда FFmpeg получил длительность → Timeline обновляет клип

```

```

QObject::connect(&ffmpeg, &FFmpegHelper::durationReady,
                 &timeline, [&timeline](QString filepath, double duration) {
    // Timeline найдёт клип по filepath и обновит duration
    emit timeline.clipDurationReady(/* clipIndex */, duration);
});

// Рендеринг
QObject::connect(&timeline, &Timeline::renderToFile,
                 &ffmpeg, &FFmpegHelper::renderTimeline);

QObject::connect(&ffmpeg, &FFmpegHelper::renderProgress,
                 &timeline, &Timeline::renderProgress);

// ===== ЭКСПОРТИРУЕМ В QML =====
engine.rootContext()->setContextProperty("cppTimeline", &timeline);

// FFmpegHelper НЕ экспортируем - он внутренний!

engine.load(QUrl(QStringLiteral("qrc:/main.qml")));

return app.exec();
}

```

---

## 🌀 ИСПОЛЬЗОВАНИЕ В QML

### main.qml:

```

` ``qml
Window {
    // ===== ПОДКЛЮЧАЕМСЯ К СИГНАЛАМ C++ =====
    Connections {
        target: cppTimeline

        // Когда клипы изменились → обновляем Timeline UI
        function onClipsChanged() {
            console.log("Timeline обновлён")
        }

        // Когда получена длительность → обновляем ширину клипа
        function onClipDurationReady(clipIndex, duration) {
            console.log("Длительность клипа", clipIndex, "=", duration, "сек")
            // Timeline.qml автоматически обновится через getClipsForTrack()
        }

        // Текущий клип изменился → обновляем VideoPlayer
        function onCurrentClipChanged(filepath, position) {
            videoPlayer.loadVideo(filepath)
            videoPlayer.seek(position)
        }

        // Прогресс рендеринга
        function onRenderProgress(percent) {
            progressBar.value = percent
        }

        function onRenderFinished(success) {

```

```

        if (success) {
            successDialog.open()
        }
    }

    function onErrorOccurred(message) {
        errorDialog.text = message
        errorDialog.open()
    }
}

// ===== ВЫЗЫВАЕМ C++ СЛОТЫ =====
Button {
    text: "Play"
    onClicked: cppTimeline.play()
}

Slider {
    from: 0
    to: cppTimeline.totalDuration
    value: cppTimeline.currentTime
    onMoved: cppTimeline.setCurrentTime(value)
}
}..

```

### Track.qml:

```

` ``qml
Rectangle {
    property int trackNumber: 1

    // ===== ПОЛУЧАЕМ КЛИПЫ ИЗ C++ =====
    property var clips: cppTimeline.getClipsForTrack(trackNumber - 1)

    // ===== ОБНОВЛЯЕМ ПРИ ИЗМЕНЕНИЯХ =====
    Connections {
        target: cppTimeline

        function onClipsChanged() {
            // Обновляем список клипов
            clips = cppTimeline.getClipsForTrack(trackNumber - 1)
        }

        function onClipDurationReady(clipIndex, duration) {
            // Repeater автоматически обновит ширину VideoClip
            clips = cppTimeline.getClipsForTrack(trackNumber - 1)
        }
    }

    // ===== DROP ВИДЕО =====
    DropArea {
        onDropped: (drop) => {
            var filepath = drop.urls[0].toString().replace(/^file:\/\//, "")
            var time = drop.x / pixelsPerSecond

            // ВЫЗЫВАЕМ C++ СЛОТ!
            cppTimeline.addClip(filepath, trackNumber - 1, time)
        }
    }
}

```

```

    }

    // ===== ОТОБРАЖАЕМ КЛИПЫ =====
    Repeater {
        model: clips

        VideoClip {
            x: modelData.startTime * pixelsPerSecond
            width: modelData.duration * pixelsPerSecond // РЕАЛЬНАЯ длительность!
            clipName: modelData.filename

            onMoved: (newX) => {
                var newTime = newX / pixelsPerSecond
                // ВЫЗЫВАЕМ C++ СЛОТ!
                cppTimeline.moveClip(modelData.id, trackNumber - 1, newTime)
            }
        }
    }
}
...

```

---

## 📺 ПОТОК ДАННЫХ

### Добавление клипа:

...

QML: User drops video

↓

QML: cppTimeline.addClip(filepath, track, time)

↓

C++ Timeline::addClip()

- └ Создает TimelineClip с duration = 0 (пока неизвестно)
- └ emit clipAdded(index)
- └ Вызывает ffmpeg.getDuration(filepath)

↓

C++ FFmpegHelper::getDuration()

- └ FFmpeg читает файл
- └ Получает duration
- └ emit durationReady(filepath, duration)

↓

C++ Timeline получает duration

- └ Находит клип по filepath
- └ Обновляет clip.duration
- └ emit clipDurationReady(index, duration)

↓

QML Connections::onClipDurationReady

- └ Track обновляет clips список

↓

QML Repeater пересоздает VideoClip

- └ VideoClip теперь с ПРАВИЛЬНОЙ шириной!

...

### Воспроизведение:

...

QML: User clicks Play

↓

QML: cppTimeline.play()

↓

C++ Timeline::play()

- └ m\_isPlaying = true
- └ emit playbackStateChanged()
- └ Timer начинает обновлять currentTime

↓

C++ Timeline::setCurrentTime(time)

- └ m\_currentTime = time
- └ emit currentTimeChanged()
- └ updateCurrentClip()
  - └ Находит клип под playhead
  - └ emit currentClipChanged(filepath, position)

↓

QML Connections::onCurrentClipChanged

- └ VideoPlayer.loadVideo(filepath)
  - └ FFmpeg декодирует кадр
  - └ Отображение в QML

...

---

**\*\*ГОТОВО! Теперь у тебя есть полная карта сигналов/слотов!\*\*** 🗺️🌟